PORTAL Studio: Research Workflow User Guide

PORTAL Contributors

This guide covers recipe construction, experiments, and visual exploration in the Python version of PORTAL Studio. It can be read and built independently of the companion manuscripts. The public PORTAL preprint [1] provides the framework background; reusable curation and augmentation workflows are documented in the accompanying repository.

Contents

1 Installation and First Launch 2

2 The PORTAL Framework 2

3 PORTAL Studio Workflow Overview 3
- 3.1 Data and File Operations . . . 3
- 3.2 Using Help in the Interface . . . 4

4 Generator Workspace 4
- 4.1 Instance Layers . . . 4
- 4.2 Component Parameters . . . 4
  - 4.2.1 Transformations . . . 6
  - 4.2.2 Randomisation Modes . . . 7
  - 4.2.3 Separate Directional Values . . . 7
  - 4.2.4 Clone, Reroll, Seeded Reroll, and Copy . . . 7
  - 4.2.5 Preparing and Saving Instances . . . 8
- 4.3 Exploratory Landscape Analysis . . . 9

5 Experiment Workspace 9
- 5.1 Task Settings . . . 9
  - 5.1.1 Realisation Types . . . 10
  - 5.1.2 Package-Backed Algorithms . . . 10
- 5.2 Pending and Finished Tasks . . . 10
  - 5.2.1 Experiment Output Measures . . . 11
- 5.3 Statistical Analysis . . . 11

6 Education Workspace 12
- 6.1 2-D Authoring View . . . 12
- 6.2 Benchmarking View . . . 13

7 Adding a Solver to the GUI 14

References 16

1 Installation and First Launch

Use Python 3.11 or newer in a dedicated environment. From the repository root, run `python -m venv .venv`. Activate it in Windows PowerShell with:

```
.\.venv\Scripts\Activate.ps1
```

On macOS or Linux, use `source .venv/bin/activate` instead. Then run:

```
python -m pip install -r requirements.txt
python -m pip check
python -m portal_studio.gui.app
```

The standard installation uses PySide6 and includes the native optimizers and the PuLP/CBC dependency for curation. To install optional NiaPy and pymoo algorithms:

```
python -m pip install -r requirements-algorithms.txt
```

PyTorch acceleration is optional; the NumPy evaluator does not require it. No MATLAB or R installation is needed. The repository README summarizes the tested environment.

The public distribution contains code and recipe JSONs, not the papers' raw feature tables, optimizer results, or experimental realizations. Studio and the runners create new results in user-controlled output directories. Use a writable checkout and start with a small FE budget.

2 The PORTAL Framework

PORTAL is a framework for constructing continuous black-box optimisation problems with controllable landscape structure. Users assemble problems with selected basin geometry, conditioning, variable interaction, ruggedness, multimodality, and block-wise composition.

The framework background is described in the public preprint [1]. This guide explains the operations available in Studio's three workspaces and identifies the separate suite-construction and augmentation runners where relevant.

PORTAL problems are constructed through three layers:

- **Layer 1 – Component.** A component is the basic basin-forming object. It defines its centre, mathematical formulation, curvature, conditioning, orientation, objective-value offset, and optional transformations.
- **Layer 2 – Landscape.** One or more components of the same dimension are combined into a landscape. When multiple components are used, the lower-envelope construction creates competing component basins across the search space.
- **Layer 3 – Block-wise composition.** Landscapes defined over different variable blocks are combined through weighted contributions to form the complete objective function. Multiple blocks allow partial separability and heterogeneous structures or contribution strengths across variable groups.

Table 1 summarises the main PORTAL terms used throughout this guide.

TABLE 1
CORE PORTAL TERMS USED IN THIS GUIDE.

Term	Meaning in the workflow
Recipe	A complete, reproducible PORTAL structural and parameter specification.
Component	The Layer 1 basin-forming object, with its own formulation, centre, shape controls, orientation, objective-value offset, and optional transformations.
Landscape	A Layer 2 function formed from one or more components of the same dimension. With multiple components, the lower-envelope construction creates competing component basins.
Block	A subset of decision variables with its own landscape and contribution weight in the Layer 3 composition.
Realisation	A mode-specific use or controlled derivation of a source recipe for evaluation.
Instance	The concrete objective landscape defined by the resulting specification and used for inspection, analysis, or optimisation.

PORTAL uses minimisation: components compete within a block, and positively weighted block landscapes are added over disjoint variable sets. This separates the blocks additively; interactions can remain within each block, including those created by component competition.

The known optimum requires valid component parameters and minimising centres inside the evaluation domain. Components can have additional minimisers, and competition can hide individual basins. Studio assigns each variable to one block, allowing all blocks to attain their minima simultaneously. If blocks shared variables, their minimising values could conflict.

3 PORTAL Studio Workflow Overview

PORTAL Studio is organised around three connected workspaces:

- **Generator:** authors PORTAL recipes and prepares concrete instances for analysis or evaluation. Users can inspect generated parameters, perform Exploratory Landscape Analysis (ELA), save instances as PORTAL JSON files, or select prepared instances in the Experiment workspace.
- **Experiment:** evaluates optimisers on PORTAL instances under reproducible run settings. It accepts instances prepared in Generator or loaded from PORTAL JSON files, supports built-in and user-added optimisers, and records repeated-run results for convergence analysis, final absolute error summaries, and statistical comparison.
- **Education:** provides interactive two-dimensional authoring, visualisation, and optimiser demonstrations. Users can edit component parameters directly, inspect components and complete landscapes, compare configurations side by side, and evaluate built-in or user-added optimisers.

For research workflows, problems are typically constructed and characterised in Generator and then evaluated in Experiment. Education provides a complementary two-dimensional environment for visual exploration, teaching, demonstration, and controlled comparison of landscape configurations.

To start Studio, install the repository requirements in the configured project environment and run the following from the repository root:

```
python -m portal_studio.gui.app
```

The GUI uses PySide6; the active environment determines the available numerical and optional solver packages. Keep the saved instance, software revision, package versions, solver settings, budgets, and seeds with research results.

For automated benchmark curation and train/test construction, use `run_suite_pipeline.py`. It performs measurement, feature preparation, de-duplication, selection, and JSON materialization. Its configuration and split controls are documented in `Recipe_Selection/suite_pipeline/README.md`. The separate `run_augmentation_dataset.py` runner creates Type 3 JSONs without running ELA or optimizers; see `Recipe_Selection/augmentation_pipeline/README.md`.

First-use walkthrough.

The following route produces a saved problem, an ELA export, and optimisation results; the later sections explain each setting.

1) In *Generator*, select + *Add Block*. Use the block-row plus control to open a new component, configure its parameters, and select + *Add Component*.
2) Select *Prepare Instance for ELA*, then *Save Instance JSON* to preserve the concrete problem.
3) In the ELA panel, tick the prepared instance, choose the sample factor and feature groups, and select *Run ELA*. Choose the Excel output file when prompted.
4) Open *Experiment*: the instance prepared in Generator automatically appears in the *Problem instance* list. Select it, or use *Load Instance* to load a saved JSON file. Choose *Source*, *Algorithm*, algorithm parameters, run count, FE budget, seed policy, and realisation type. Select + *Add Task*.
5) Tick the task in *Pending Tasks*, choose the pool and workers, and select *Run Selected Tasks*.
6) Check the terminal state and error summaries in *Finished Tasks*. Select the completed task in *Statistical Analysis* for *Trend Plots* or *Box Plots*; statistical tests require multiple compatible completed tasks.

3.1 Data and File Operations

PORTAL Studio supports several forms of inspection, modification, persistence, and export across the three workspaces. Table 2 summarises their roles.

TABLE 2
DATA OPERATIONS AND FILE ROLES IN PORTAL STUDIO.

Operation group	Role in the workflow
Inspection	Generated-parameter tables, rotation tables, ELA outputs, finished-task summaries, statistical-test tables, convergence plots, and box plots expose values produced during construction and evaluation.
Modification	Reroll and seeded-reroll actions resample generated values; copy controls reuse compatible realised values; clone creates an independently editable component duplicate.
Persistence	PORTAL JSON files store concrete benchmark problems. Finished-result ZIP archives preserve task settings, run errors, convergence data, and analysis identities for later import. SQLite result bundles store the underlying measurements.
Analysis and export	ELA feature tables can be exported to Excel; Statistical Analysis plots can be exported as PNG, PDF, or SVG; Education plots can be saved as PDF.

Keep the instance JSON for later problem evaluation. To reopen completed experiments for analysis in a later session, export and import finished-result archives as described in Section 5.2.

3.2 Using Help in the Interface

PORTAL Studio provides contextual help throughout the interface. The question-mark *Help* button in the toolbar starts a guided tour of the main workspaces (Figure 1), while nearby question-mark controls explain individual parameters, tables, and settings.

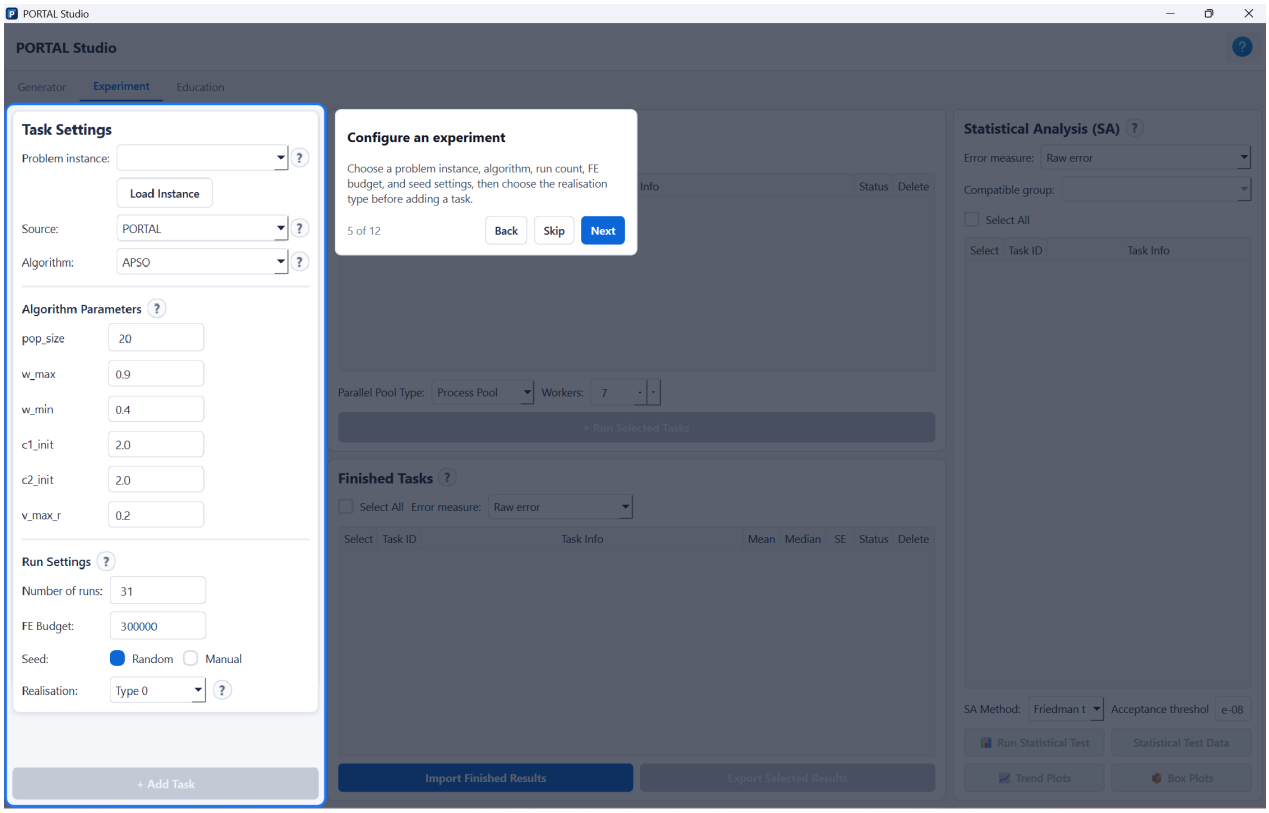


Fig. 1. Guided tour highlighting Task Settings in Experiment. The guide explains the selected panel and provides Back, Skip, and Next controls.

4 Generator Workspace

The Generator workspace is the main authoring environment for constructing PORTAL recipes and preparing concrete instances for analysis. As shown in Figure 2, it contains three panels: the left panel defines the block/component hierarchy, the middle panel edits the selected block or component, and the right panel analyses prepared instances using ELA.

4.1 Instance Layers

The *Instance Layers* panel represents the current recipe as a block-and-component tree. Select + *Add Block* to create a block, initially with dimension 30 and weight 1. Each block must have at least two dimensions, as specified in Part I.

Select a block to edit its dimension and weight in the middle panel's *Block Parameters*, then commit with *Apply Changes*. Changing a populated block's dimension prompts for confirmation and regenerates all component values, including rotations and transformations, from the stored generation specifications and effective random streams. Existing coordinates are regenerated rather than simply extended. Inspect the new values and prepare a new instance after applying the change.

Use the plus control on a block row to open a new component editor. Configure the fields in *Component Parameters*, then select + *Add Component* to store it in that block. To edit an existing component, select its tree row, change the fields, and select *Apply Changes*. Components within a block share its dimension and combine through the lower-envelope construction. Component-row controls open generated parameters, clone the component, or delete it.

4.2 Component Parameters

The *Component Parameters* panel defines the selected Layer 1 component. Choose its base form, location, shape, rotation, and optional transformations, then commit the settings as described above.

The *Base form* selector chooses between PORTAL's two component formulations. *Base Form A* corresponds to the per-direction exponent formulation (Form 1 in Part I), while *Base Form B* corresponds to the shared single-exponent formulation (Form 2).

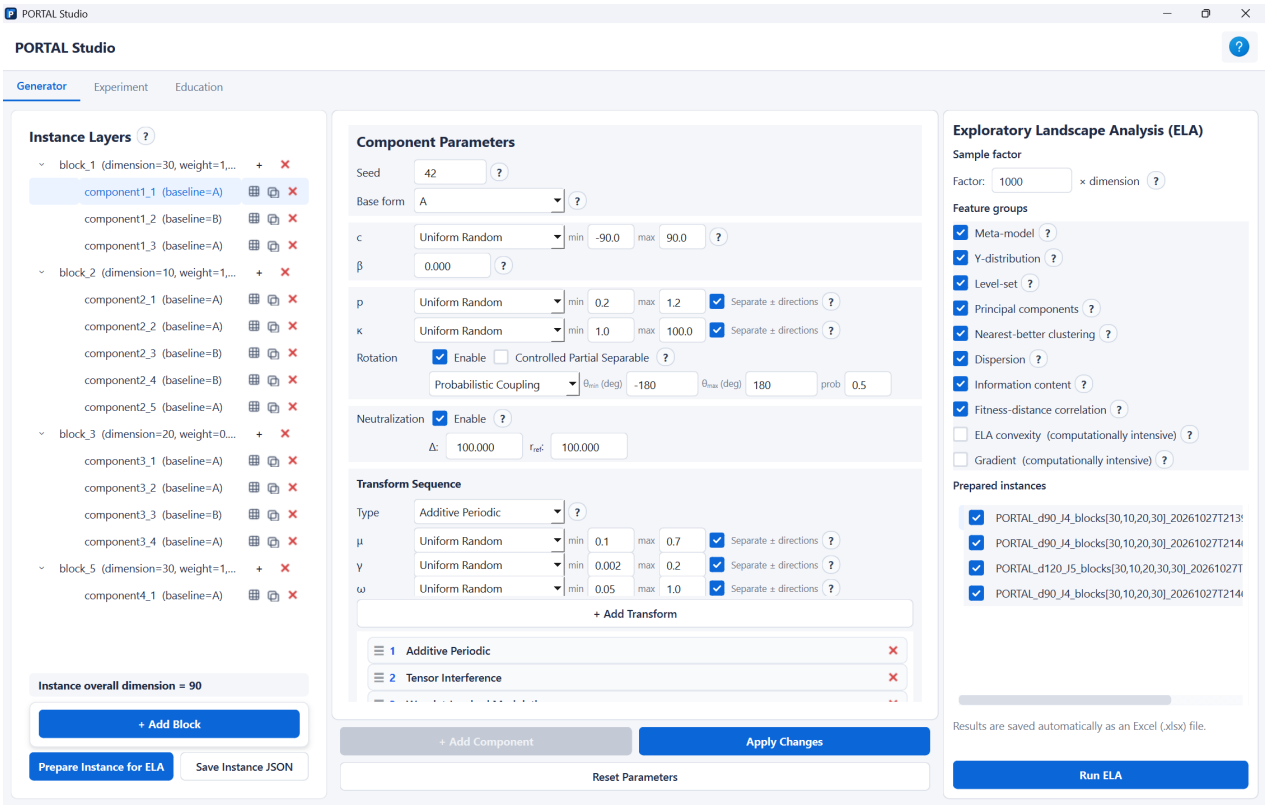


Fig. 2. Generator workspace with the block/component hierarchy on the left, component settings in the middle, and exploratory landscape analysis on the right. Question-mark controls provide contextual help.

For either formulation, users can configure the component's centre, curvature, anisotropy, objective-value offset, rotation, and transformations. Each setting is briefly described below; the framework preprint [1] gives the mathematical construction.

The exponent p controls basin curvature and growth away from the component centre. The setting $p = 1$ gives the quadratic reference case; values below and above 1 produce subquadratic and superquadratic growth, respectively. The linear-growth reference is $p = 0.5$; lower values give sublinear growth and higher values give superlinear growth.

The anisotropy factors κ control directional scaling. Equal κ values give equal directional scaling, while a higher ratio of the largest to the smallest factor produces greater directional contrast. Even with equal anisotropy factors, unequal curvature exponents can produce different growth profiles across directions.

For component k, Base Form A assigns separate curvature exponents $p^{+}_{k,i}$ and $p^{-}_{k,i}$ to the positive and negative sides of each coordinate i. Base Form B uses one exponent p_k for the entire component, while retaining directional anisotropy through $\kappa^{\pm}_{k,i}$.

The centre c controls the component location in the search space, while the vertical offset β sets its objective value at that centre. Within a multi-component block, the components compete through the lower-envelope construction. In a multi-block instance, the block-level landscapes are combined using their contribution weights.

Enable *Neutralization* to reduce unintended scale differences when changing curvature or anisotropy. Set r_{ref} as the reference displacement along each coordinate and Δ as the target reference rise above the component's centre value. This calibration preserves directional differences.

After changing curvature exponents, anisotropy factors, or the block dimension, apply the edits and prepare a new instance. Studio automatically recalculates the calibration factors ρ when *Neutralization* is enabled; otherwise, it sets them to one. The interface retains the label *Neutralization* for this calibration control.

Rotation sets the orientation of the component's internal coordinate system through pairwise angles. Leave *Enable* unchecked for *No Rotation*. Otherwise, enable rotation and choose a mode from the dropdown, or tick *Controlled Partial Separable* and select *Configure*. All angle inputs and rotation tables use degrees. Table 3 summarises the modes.

TABLE 3
ROTATION MODES AVAILABLE IN THE GENERATOR WORKSPACE.

Mode	Interpretation
No Rotation	Sets all pairwise rotation angles to zero, leaving the component axes aligned with the coordinate axes.
Fully Connected	Assigns an angle to every variable pair, producing dense coordinate mixing.
Probabilistic Coupling	Selects each variable pair according to a user-defined probability; only selected pairs receive rotation angles.
Uniform Random Angle	Applies one sampled angle to all variable pairs.
Fixed Angle	Applies one user-specified angle to all variable pairs.
Chain-like Coupling	Specifies angles only for neighbouring coordinate pairs; combining these rotations can indirectly mix non-neighbouring coordinates.
Controlled Partial Separable	Divides variables into groups and applies rotation only within groups, leaving cross-group rotation entries zero.

For *Controlled Partial Separable*, the *Configure* dialog (Figure 3) specifies automatic or manual group sizes and the within-group angle policy. Coordinates are randomly assigned to groups of the requested sizes. Rotation mixes coordinates only within each group; any unassigned coordinates remain unchanged by the rotation. These groups control rotation only. The component formulation, transformations, and competition between components can introduce additional variable interactions.

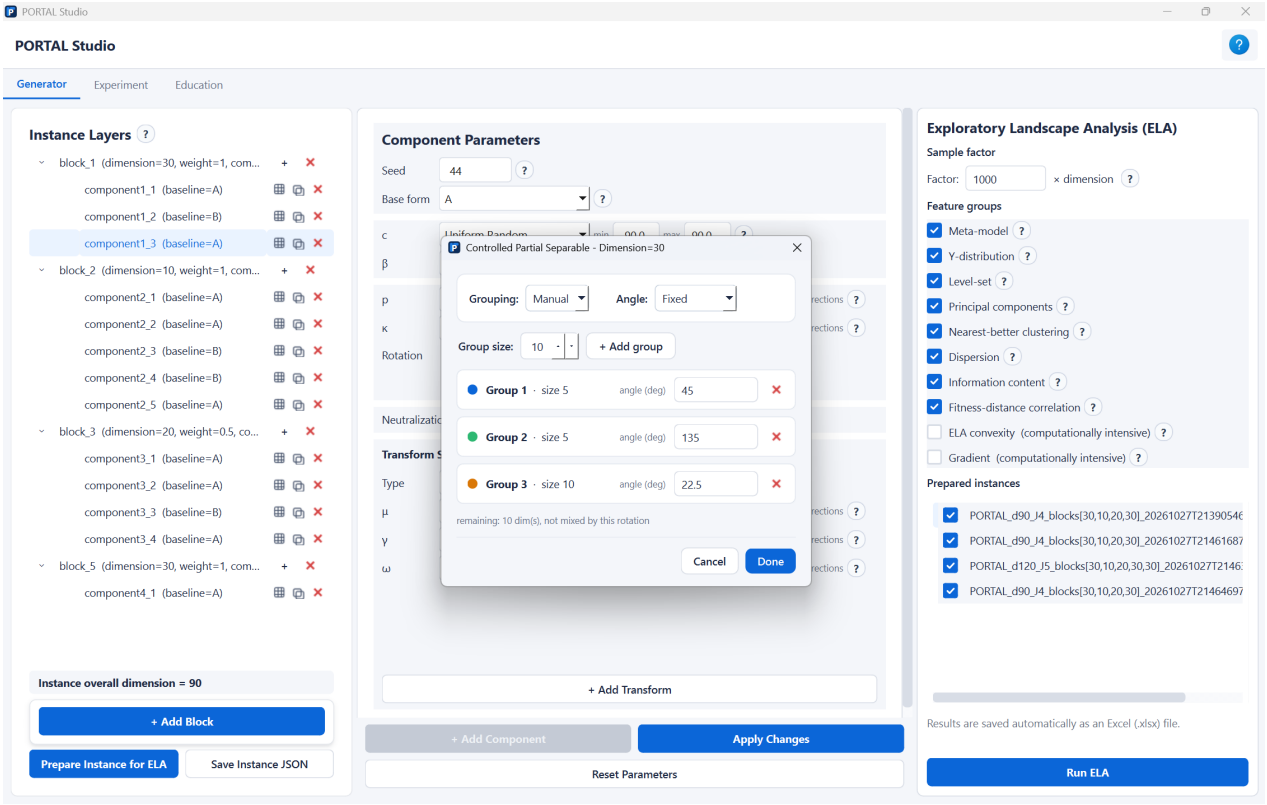


Fig. 3. Manual rotation grouping in a 30-dimensional component. Groups of sizes 5, 5, and 10 use fixed angles of 45°, 135°, and 22.5°, respectively; the remaining 10 dimensions are not mixed by this rotation.

4.2.1 Transformations

Transformations add controlled ruggedness and local structure to a component. A component without transformations retains its baseline basin geometry, while transformed components can exhibit repeated oscillations, localised ruggedness, cross-coordinate interference, or radial structures.

PORTAL distinguishes element-wise and coupling transformations. Element-wise transformations act independently on each internal coordinate after rotation. Coupling transformations depend jointly on multiple internal coordinates and can introduce additional non-separability.

Multiple transformations can be applied sequentially to the same component. Their order is part of the recipe and can affect the resulting landscape. Table 4 summarises the transformation operators available for component-level ruggedness control.

TABLE 4
TRANSFORMATIONS AVAILABLE FOR COMPONENT-LEVEL RUGGEDNESS CONTROL. THEIR PARAMETERS CAN BE EXPLORED VISUALLY IN THE EDUCATION WORKSPACE.

Transformation	Effect on the component
Additive Periodic	Adds regular coordinate-wise oscillations, producing repeated local structure.
Log-Sinusoidal	Introduces scale-sensitive oscillation in logarithmic coordinate space.
Wavelet-Inspired Modulation	Adds localised oscillations along each internal coordinate, with extent controlled by an envelope.
Tensor Interference	Introduces cross-coordinate interference and structured coupling.
Radial Polynomial-Trigonometric	Applies radial modulation based on distance from the centre, introducing concentric structure.

To add a transformation in Generator, choose its *Type* in the *Transform Sequence* area, configure the displayed parameters, and select + *Add Transform*. Drag a sequence row to reorder it, or use its cross control to delete it. For an existing component, commit sequence changes with *Apply Changes*; for a new component, use + *Add Component*. To edit an existing transformation, select its row, change the settings, and select *Apply Changes*. The component stores transformations, parameters, and order together. Only transformations added to the sequence are applied.

Repeated transformation types: Each transformation keeps its own settings and is applied in the listed order, even when the same type is used more than once.

4.2.2 Randomisation Modes

Most Generator parameters use a generation mode together with a range or mode-specific setting. The selected mode determines how parameter values are generated across dimensions and, where applicable, across positive and negative directions (see Section 4.2.3).

TABLE 5
RANDOMISATION MODES AND THEIR DEFINITIONS. AVAILABLE MODES DEPEND ON THE SELECTED PARAMETER.

Mode	Definition
Uniform Random	Samples each coordinate value independently and uniformly from the specified range.
With Exclusion Zone	Samples centre coordinates outside an excluded interval placed inside the centre sampling range, choosing the lower or upper allowed interval with equal probability.
Fixed Value	Uses the user-specified exponent in every coordinate.
Fixed Gradient	Assigns evenly spaced exponents from the minimum to the maximum in coordinate order.
Beta Distribution	Samples values from a beta distribution mapped to the specified range. Shape parameters $\alpha, \beta > 0$ give a symmetric distribution when equal; unequal values favour one end.
Linear Spread (Shuffled)	Generates evenly spaced values across the specified range and randomly permutes their coordinate assignments.
Isotropic Random	Samples one anisotropy factor and uses it for every coordinate of the component.
Fixed Unit Scale $(\kappa = 1)$	Sets every anisotropy factor to one.
Dimension-shared	Samples one value for a transformation parameter and uses it across all coordinates of the component.

4.2.3 Separate Directional Values

Parameters with positive and negative directional forms use matched values by default: the negative-side values are copied from the corresponding positive-side values.

Where available, *Separate* $\pm$ *directions* generates the two signs independently from the shared generation mode and sampling bounds. This allows directional asymmetry in curvature, conditioning, or transformation strength.

4.2.4 Clone, Reroll, Seeded Reroll, and Copy

Open a component's generated-parameter table using its table icon in *Instance Layers*. *Reroll* is shown initially; tick *Seeded reroll* or *Copy* above the table to reveal those action columns. The available actions are:

- *Clone* copies a component into a selected block. The clone is independently editable and is regenerated to match the target block dimension when necessary.
- *Reroll* regenerates one parameter row using its generation mode and settings. A matched negative-side row follows a rerolled positive-side row. Rerolling a copied value (see *Copy* below) removes its copy override, so it is generated independently of the source component.

- *Seeded reroll* regenerates a parameter row using a user-specified seed. The same seed reproduces the same values when applied to a compatible parameter row with the same generation settings.
- *Copy* takes a one-time snapshot of a parameter's current value from another component with the same dimension and base form. Transformation parameters additionally require the same transformation type at the same sequence position. Repeat *Copy* to resynchronise after source edits; copied values do not update automatically.

Use Table 6 to choose an action. Figure 4 shows the destination selection for a cloned component.

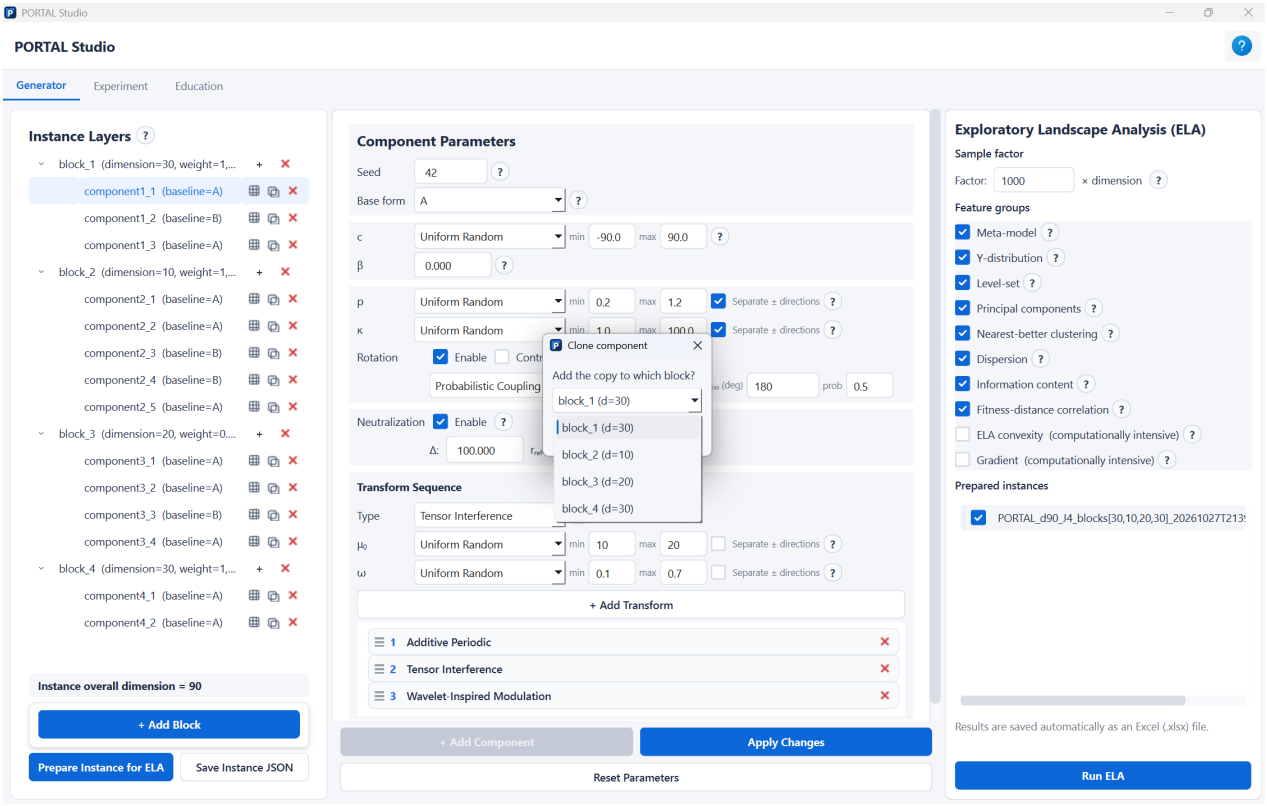


Fig. 4. Clone-component dialog for choosing the destination block.

TABLE 6
CHOOSING AMONG CLONE, REROLL, SEEDED REROLL, AND COPY. COMPONENT A HAS ALREADY BEEN CREATED. THE SCENARIOS BELOW SHOW HOW TO CREATE COMPONENT B FROM A, OR CHANGE B AFTER IT HAS BEEN CREATED.

What you want	Action	Result
Create B as a complete copy of A.	**Clone A**	Creates B with A's settings and values. Choose a destination block with the same dimension to retain the values.
Generate another centre for B using B's current sampling settings.	**Reroll B's centre**	Generates another centre without asking you to enter a seed.
Generate B's centre using a seed you choose.	**Seeded reroll B's centre**	Reproduces the same centre when that seed, dimension, and generation settings are reused.
Give B the same centre as A, keeping B's other settings.	**Copy A's centre onto B**	Copies A's current centre. Later changes to A do not update B automatically.

Each parameter normally uses the component seed. *Seeded reroll* assigns a separate seed to the selected parameter. To return to the component seed, open the generated-parameter table, tick *Seeded reroll*, then right-click the parameter's button in the *seed* column and choose *Clear row seed*. This regenerates its values. Because rerolls, separate seeds, and copied parameters affect the final values, save the instance JSON with your results to preserve the exact problem evaluated.

4.2.5 Preparing and Saving Instances

Each block must contain at least one component. Apply any pending edits with *Apply Changes*, then select *Prepare Instance for ELA*. The prepared instance appears in the ELA panel and becomes available in Experiment without saving and reloading.

Select *Save Instance JSON* and choose a filename and folder. After editing the problem, apply the changes and prepare the instance again. You can then save it under the same filename to replace the previous version, or choose a different name to keep both. To use the saved problem in a later session, select *Load Instance* in Experiment and choose the JSON file.

4.3 Exploratory Landscape Analysis

The Exploratory Landscape Analysis panel computes landscape features using `pflacco` [2]. Select one or more prepared instances, set the sample factor, and choose the feature groups. The initial sample size is the factor multiplied by the instance dimension.

The sample factor defaults to 1000 and accepts values from 10–10,000. Sampling uses an improved Latin-hypercube design over the instance bounds with a fixed internal seed of 42. The default initial design for $d = 30$ therefore contains 30,000 points.

The ten groups are meta-model, response distribution, level-set, principal components, nearest-better clustering, dispersion, information content, fitness-distance correlation, ELA convexity, and gradient. The computationally intensive convexity and gradient groups are initially unchecked; enable them for protocols that use all ten.

Select *Run ELA*, confirm the export prompt, and choose an Excel output file. Studio writes the feature values when processing finishes. *Stop ELA* requests cancellation; a cancelled analysis saves no ELA file. Before comparing instances, check the `failed_groups` column to identify feature groups that could not be calculated. Leave missing results as missing values; replacing them with zero would incorrectly treat them as measured results.

ELA estimates depend on sample size, design, implementation, and seed. Use consistent settings and versions across comparisons. Exports contain raw descriptors; use the separate suite runner for preprocessing, redundancy assessment, and performance-based curation. Its feature-preparation settings are described in `Recipe_Selection/suite_pipeline/README.md`.

5 Experiment Workspace

The *Experiment* workspace lets users evaluate and compare optimisation algorithms on PORTAL problems. Users can configure repeated runs, examine errors and convergence, and perform statistical comparisons.

As shown in Figure 5, the workspace is organised into *Task Settings*, *Pending Tasks*, *Finished Tasks*, and *Statistical Analysis*.

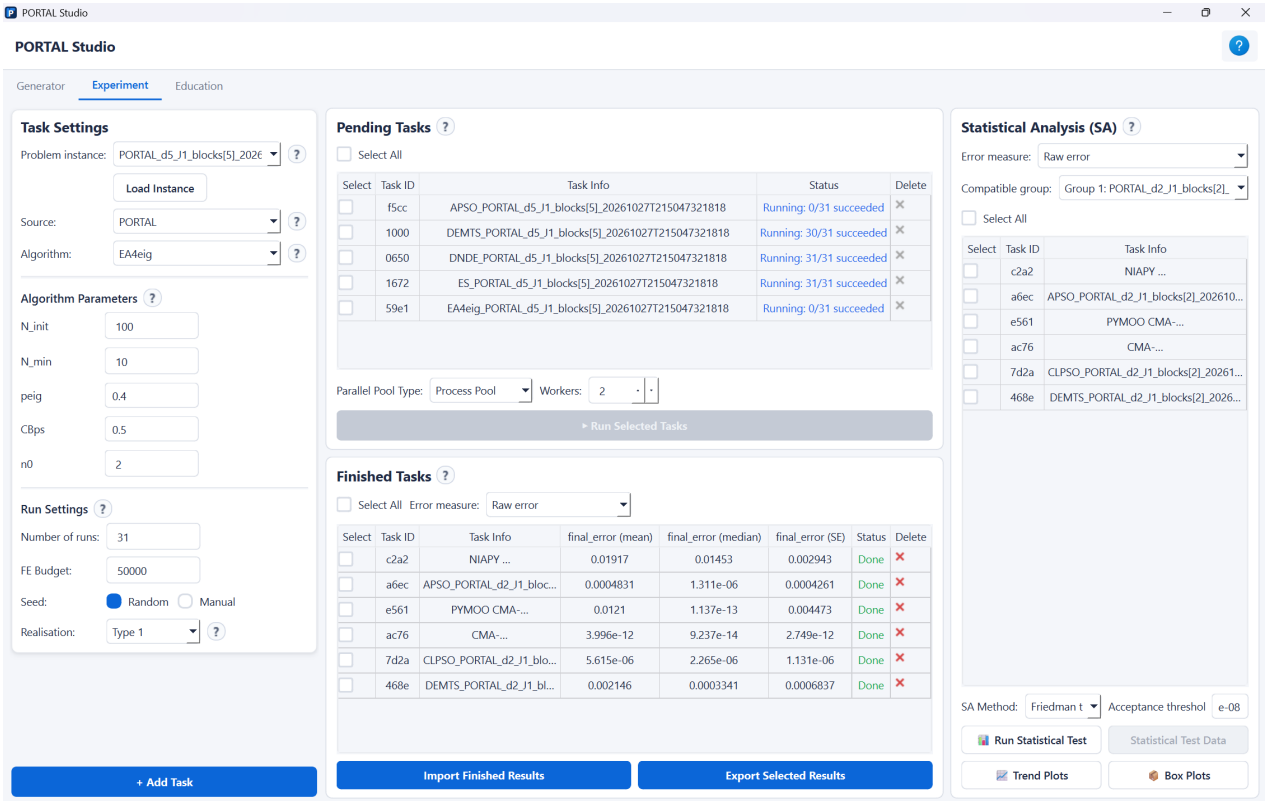


Fig. 5. Experiment workspace with Task Settings on the left, Pending and Finished Tasks in the middle, and Statistical Analysis on the right. The selected problem in Task Settings can differ from the compatible group selected for analysis.

5.1 Task Settings

The *Task Settings* panel defines an experiment task before it is added to the pending queue.

The *Problem instance* row selects the PORTAL instance to be evaluated. Instances prepared in Generator automatically appear in this list when the Experiment workspace is opened. Use *Load Instance* to load a saved PORTAL JSON file.

Algorithm selection uses two linked controls. First, *Source* selects the implementation group: native PORTAL solvers or an integrated third-party package. The *Algorithm* row then lists the optimisers available for that source, and the *Algorithm Parameters* area updates to show the parameters exposed by the selected solver.

The *Run Settings* area specifies the number of repeated runs, function-evaluation budget, algorithm seed policy, and realisation mode. *Number of runs* gives the number of optimiser executions requested for the task, while *FE Budget* sets the maximum objective-function evaluations per run.

The *seed policy* controls the optimiser's random choices across runs, such as initial candidate solutions. *Random* assigns seeds automatically. *Manual* uses the entered *Algorithm seed* for the first run and adds one for each subsequent run; for example, entering 42 gives seeds 42, 43, 44, and so on.

The initial run count is 31, with a range of 1–200. The FE budget initially follows $10{,}000d$, where d is the selected instance dimension, within the allowed range of 100–10,000,000 evaluations. Once you edit the budget, changing the selected instance no longer updates it automatically. Choose and record settings appropriate to the study. Confirm that all requested runs completed successfully before treating a task as a complete repeated-run sample.

5.1.1 Realisation Types

The *Realisation* setting determines whether repeated runs use the stored problem or controlled variants of it. Type 0 keeps the problem unchanged for every run; Types 1 and 2 generate a variant for each run.

Type 1 changes component locations and shifts objective values while retaining shape and interaction settings. Type 2 also rearranges existing curvature, anisotropy, and signed transformation parameter values instead of drawing new values for those parameters. Either mode can change optimisation difficulty, since moving components within fixed search bounds can affect the search even when shape settings are unchanged. Landscape measurements may also change, including in response to objective shifts. Table 7 summarises the three modes.

TABLE 7
REALISATION TYPES AVAILABLE IN THE EXPERIMENT WORKSPACE. TYPE 3 AUGMENTATION IS PROVIDED SEPARATELY BY THE DATASET RUNNER.

Type	**Experiment meaning**
Type 0	Evaluates the same stored landscape in every run. Different algorithm seeds can still produce different search behaviour.
Type 1	Within each block, applies the same bounded translation to all component centres and the same shift to their objective-value offsets. Shape and interaction parameters, distances between centres, and differences between offsets are preserved within the block.
Type 2	Within each block, attempts an orthogonal move of the centre configuration, followed by a bounded translation and a common objective-value offset shift. Existing signed parameter pairs are permuted and may swap directions; existing block weights are reassigned for multi-block recipes. Neutralisation factors are recalculated.

To compare algorithms on equal terms, each should face the same problem variants. Studio therefore uses fixed realisation settings so that, for the same source problem and realisation type, corresponding runs evaluate matching variants.

Once the problem, algorithm, parameters, run settings, and realisation mode are configured, select + *Add Task* to add the task to *Pending Tasks*.

5.1.2 Package-Backed Algorithms

PORTAL Studio supports optimisers from NiaPy [3] and single-objective algorithms from PyMOO [4]. Package-backed algorithms expose a small common set of GUI parameters, typically population size, so that they can be used consistently alongside PORTAL's native solvers. The corresponding Python package must be installed in the active environment before the algorithm can be run.

5.2 Pending and Finished Tasks

Tasks created in *Task Settings* appear in *Pending Tasks*. Tick one or more tasks, choose *Process Pool* and the number of *Workers*, or *Serial (1 worker)*, then select *Run Selected Tasks*. Each task expands into the requested repeated runs.

When a task completes, fails, or is cancelled, it moves to *Finished Tasks*, which reports its summary and terminal state. Successfully completed tasks become available in Statistical Analysis. Deleting a finished task removes its row without deleting the on-disk bundle.

Saving and reopening finished results.

Studio automatically saves run results under `results/raw_runs/batch_<timestamp>/`, relative to the folder from which Studio was launched. To keep selected finished tasks in a portable file for later analysis, tick their rows in *Finished Tasks*

and select *Export Selected Results*. The ZIP archive preserves each task's algorithm settings, problem-source information, budget, seeds, realisation mode, status, recorded run errors, convergence checkpoints, and identities used for statistical pairing. Tasks retain their individual settings even when exported together. The archive also includes recorded run errors in `run_errors.csv` and convergence data in `convergence.csv`; both files can be opened in Excel.

Select *Import Finished Results*, then choose the exported results ZIP file. The saved tasks are restored to *Finished Tasks* without rerunning the experiments. Successfully completed tasks become available in Statistical Analysis.

5.2.1 Experiment Output Measures

In *Finished Tasks*, the *Error measure* selector offers the three measures summarised in Table 8. Raw error is selected by default. For whichever measure is selected, Studio reports the *mean*, *median*, and *standard error (SE)* across runs.

TABLE 8
ERROR MEASURES AVAILABLE IN FINISHED TASKS.

Measure	Meaning
Raw error	The absolute difference between the best objective value found and the evaluated instance's known optimum value, as defined in equation (1).
Normalised error	The raw error divided by the evaluated instance's reference scale.
Reference-scale error (%)	The normalised error multiplied by 100.

For run r, the raw final error is

$$E_{\mathrm{final},r} = \left| B_{r,T} - F_r^{\star} \right|, \tag{1}$$

where $B_{r,T}$ is the best objective value found by the end of the evaluation budget T, and $F_r^{\star}$ is the known optimum value of the instance evaluated in that run.

For example, a run with raw error 6 and reference scale 120 has a normalised error of 0.05, equivalent to a reference-scale error of 5%. Reference-scale percentages can exceed 100%.

Normalised and percentage measures require enabled calibration for every component and a recorded reference scale. When these requirements are not met, *Finished Tasks* displays a dash for those measures; raw results remain available. For blocks b with positive weights w_b, component offsets $\beta_{b,k}$, and declared calibration rises $\Delta_{b,k}$, the nominal scale is

$$S_{\mathrm{ref}} = \sum_b w_b \left[\max_k (\beta_{b,k} + \Delta_{b,k}) - \min_k \beta_{b,k} \right]. \tag{2}$$

It describes declared reference levels, not the attained objective range or problem difficulty. The value is computed from the concrete instance used in each run.

5.3 Statistical Analysis

The *Statistical Analysis (SA)* panel compares results from finished tasks. It provides statistical tests of final errors, convergence plots, and final-error distributions. Start by choosing the error measure and a group of compatible tasks, then select the tasks to analyse.

Choosing a measure and group.

Select *Raw error*, *Normalised error*, or *Reference-scale error (%)* from *Error measure*. This selection applies to both tests and plots. Normalised and percentage measures require a valid recorded reference scale for every run; tasks without one are excluded from those measures but can remain available under Raw error.

The *Compatible group* menu updates automatically for the selected measure. Each group contains successfully completed tasks with valid run data and matching:

- **Recorded problem identities:** corresponding runs evaluated the same source problem and concrete problem variants.
- **Realisation type:** the tasks used the same realisation mode.
- **Evaluation budget and run count:** the tasks used the same FE budget and requested number of runs.

Different algorithms, algorithm parameters, and algorithm seeds are allowed within a group. For example, PSO and DE can be compared when their corresponding runs evaluate matching Type 1 variants under the same budget.

Choose a group, then tick the task rows to include, or use *Select All*. Tasks with different problem identities or trial settings appear in separate groups. Changing the error measure rebuilds the groups and clears the selected rows and previous test results.

Running a statistical test.

Each selected task must contain at least two completed runs. Comparisons are paired by run index: run 1 of one task is compared with run 1 of the other tasks on the matching problem, and likewise for subsequent runs. Choose the test from *SA Method*:

- **Friedman test:** compares at least three selected tasks using their per-run final errors in the chosen error measure. If the overall test is significant, Studio reports pairwise Wilcoxon signed-rank comparisons with Holm correction.
- **Wilcoxon signed-rank:** select two tasks for a direct paired comparison. If more tasks are selected, Studio reports all pairwise comparisons with Holm correction.

Select *Run Statistical Test*, then *Statistical Test Data* to view the result tables. Studio checks the recorded data for validity and compatible pairing before testing. The significance level is fixed at $\alpha = 0.05$; pairwise significance is determined from the Holm-adjusted p-values. An `n/a` entry means that a comparison is undefined, for example because its paired error samples are identical.

Viewing convergence and final errors.

Use *Trend Plots* or *Box Plots* for the selected tasks; a statistical test is not required first. Trend plots show the mean best-so-far error across runs, with a band of one standard error on either side, as illustrated in Figure 6. Box plots show the distribution of final errors. Both use the selected error measure, and a task with one completed run can be plotted.

The *Acceptance threshold* marks a target error as a reference line on the trend plot. Enter it in the selected measure's units; changing the measure clears the previous threshold. This target is separate from the statistical significance level. Use *Export* in a plot window to save PNG, PDF, or SVG.

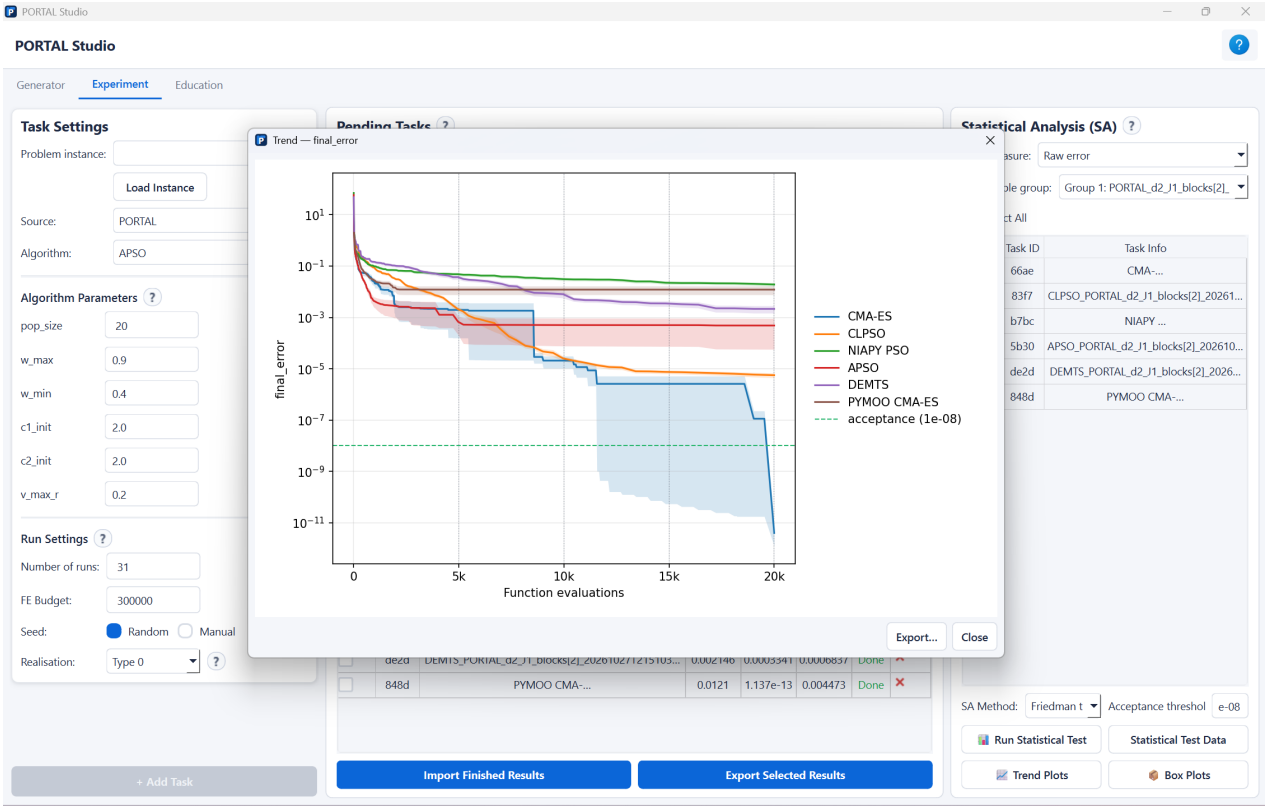


Fig. 6. Raw-error convergence for six algorithms on a two-dimensional, single-block source problem, using 31 Type 1 runs per task and a budget of 20,000 function evaluations. Curves show across-run means with bands of one standard error. The error axis uses a logarithmic scale; the dashed green line marks the acceptance threshold of 10^{-8}.

6 Education Workspace

The **Education** workspace provides an interactive two-dimensional environment for exploring PORTAL landscape construction and optimiser behaviour. Users can modify component parameters, observe their effects on the landscape, and examine how optimisers move and converge during search.

The workspace contains two connected views: **2-D Authoring**, where components and landscapes are constructed and visualised, and **Benchmarking**, where the resulting landscapes are used for optimiser demonstrations and comparison.

6.1 2-D Authoring View

The **2-D Authoring** view (Figure 7) lets users build landscapes in two variables and inspect how component settings affect their shape. Parameters are entered directly for each coordinate, and rotation angles use degrees.

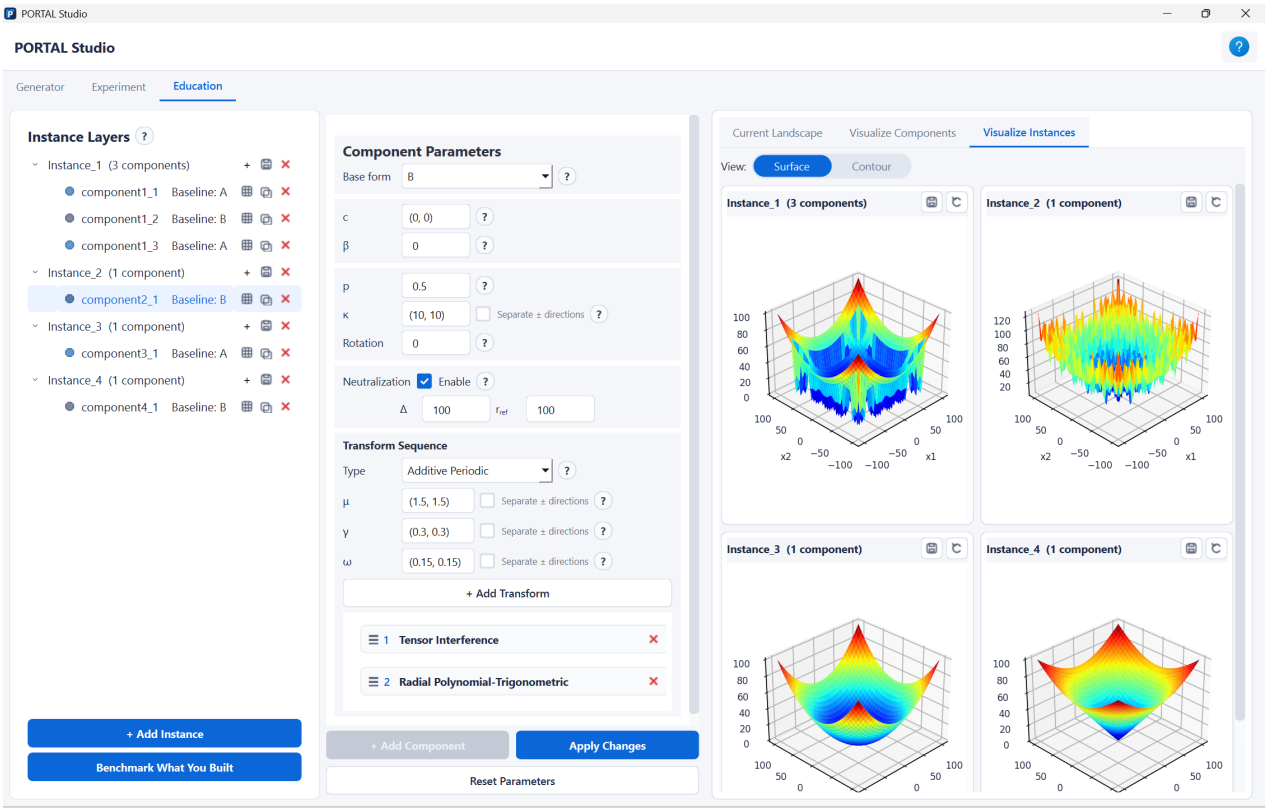


Fig. 7. Education's 2-D Authoring view. Instance Layers appear on the left, component settings in the middle, and the *Visualize Instances* surface comparison on the right.

Create and edit an instance.

Select **+ Add Instance**, then use the instance row's plus control to open a component editor. Enter its parameters and select **+ Add Component**. Repeat to add more components. To edit an existing component, select its row, change the settings, and select **Apply Changes**.

Inspect the landscape.

Select a component to view it individually, or select the instance to view the combined landscape. **Current Landscape** provides surface and contour plots; **Visualize Components** and **Visualize Instances** provide side-by-side comparisons.

Add transformations.

Choose a transformation *Type*, configure its parameters, and select *+ Add Transform*. Drag sequence rows to reorder them or use their cross controls to remove them. Save sequence changes with *Apply Changes* for an existing component or *+ Add Component* for a new one.

To edit a transformation, select its row, change its parameters, and select *Apply Changes*. For an existing component, select *Apply Changes* again to save the updated component. To explore transformation order, clone a component and reorder its transformations while keeping the other settings unchanged. Use *Visualize Components* to compare their surfaces or contours.

Save your work.

Use the instance row's *Save this instance as JSON* icon to save the problem for later use in Experiment. Plot save icons export individual figures as PDF.

6.2 Benchmarking View

The **Benchmarking** view shows how an optimiser searches an authored landscape and how its error changes during the run. Select **Benchmark What You Built** to open this view; use the toolbar back arrow to return to **2-D Authoring**.

Run an optimiser.

Select the problem instance, choose a *Source* and *Algorithm*, and configure the algorithm parameters. These controls work as in Experiment (Section 5.1). Set *Budget (FE)* and select *Run*. The running dialog's *Cancel* control requests cancellation during computation.

The initial budget is 20,000 evaluations for a two-dimensional problem, with an allowed range of 10–1,000,000. Education uses algorithm seed 42. The *Acceptance rate* field sets the target absolute error shown as a reference line on the convergence plot; its default is 10^{-8}.

Compare two setups.

Enable **Comparison mode** to display Setup A and Setup B. Choose an instance, source, algorithm, and parameters for each setup, then select *Run*. Both setups use the same budget and acceptance threshold. Their recorded runs are replayed together at matching evaluation counts, as illustrated in Figure 8. Use Experiment for comparisons over repeated runs and statistical analysis.

Replay the results.

Playback starts automatically after computation finishes. Search positions appear over the landscape, while the convergence plot shows the best-so-far absolute error relative to the evaluated instance's known optimum. Use *Play*/*Pause* to control playback, *Reset* to return to the start, *Speed* to change the playback rate, or drag the FE slider to inspect a recorded evaluation. These controls replay the recorded run without running the optimiser again.

Surface and contour plots use a finite sampling grid, so very narrow landscape features may not be visible.

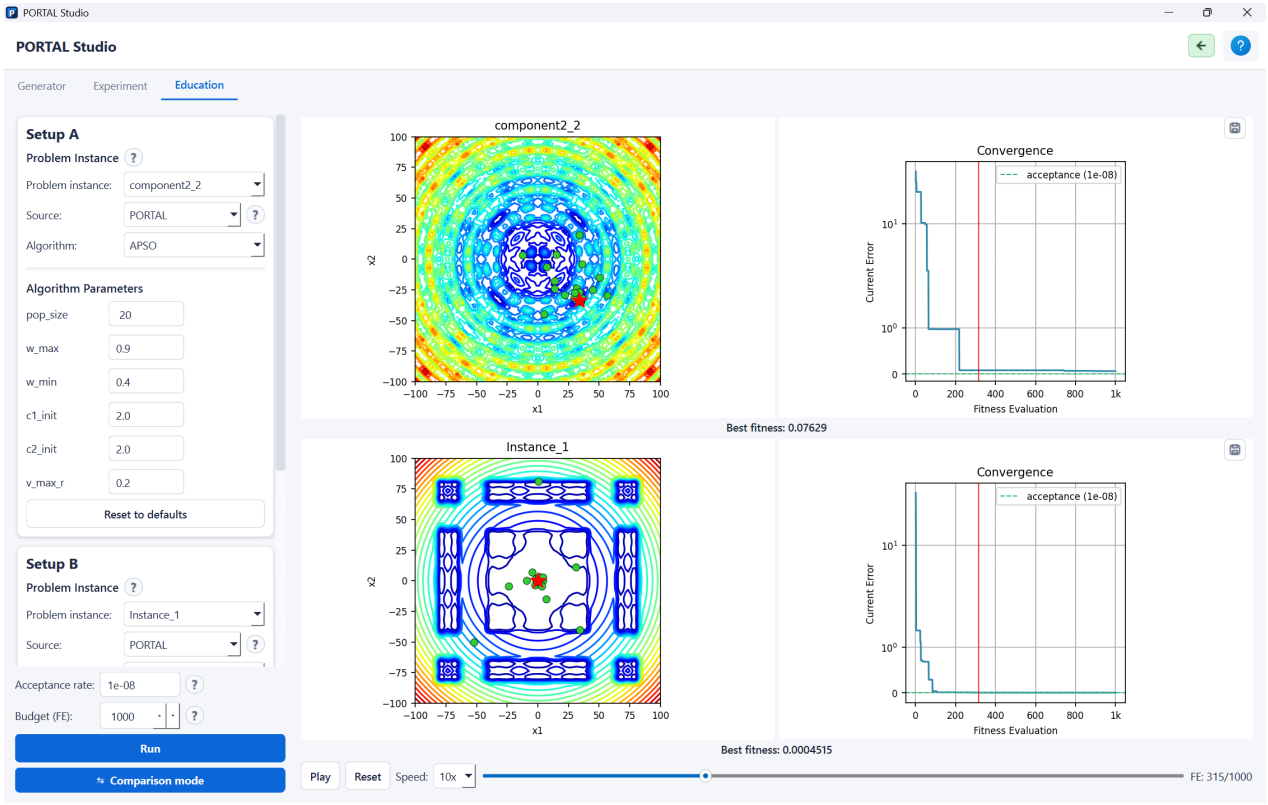


Fig. 8. Education Benchmarking in comparison mode. The two setups show search positions over contour plots and their best-so-far error curves. Playback is paused at evaluation 315 of a 1,000 FE budget; dashed green lines mark the acceptance threshold of 10^{-8}.

7 Adding a Solver to the GUI

You can add your own optimisation algorithm to Studio by creating a Python wrapper in the repository's `algorithms/` directory. Studio discovers these wrappers at startup and makes them available in Experiment and Education Benchmarking.

1. Create the wrapper.

Create a file such as `algorithms/MySolver.py`. Define a class that inherits from `BaseAlgorithm` and implements `run(problem)`. The `problem` object provides the dimension, search bounds, evaluation methods, and budget check.

The following minimal example samples candidates uniformly within the search bounds. Replace its sampling step with your algorithm's search procedure.

```
"""MySolver -- Minimal example of a PORTAL solver wrapper."""

import numpy as np
from base_algorithm import BaseAlgorithm


class MySolver(BaseAlgorithm):
    def __init__(self):
        super().__init__(name="MySolver")

    def run(self, problem):
        while not problem.is_done():
            x = np.random.uniform(
                problem.lb, problem.ub, size=problem.d
            )
            value = problem.evaluate(x)
            # Use value to update your algorithm's search state.

        return self._make_result(problem)
```

Evaluate every candidate through `problem.evaluate(x)`, or use `problem.evaluate_batch(X)` for batches of candidates. These methods count function evaluations and update the recorded best solution. Check `problem.is_done()` before requesting further evaluations, and return `self._make_result(problem)` when the run ends.

2. Expose the algorithm's parameters.

Add configurable parameters to the class constructor, give them suitable defaults and type annotations, and store them for use in `run`. For example, a population-based solver might define:

```
def __init__(self, pop_size: int = 30):
    super().__init__(name="MySolver")
    self.pop_size = pop_size
```

Studio reads the constructor arguments to create the **Algorithm Parameters** controls. To specify input ranges, step sizes, tooltips, or categorical choices, add an entry under the solver's displayed name in `PARAM_OVERRIDES` in `portal_studio/algorithms/solver_params.py`. Existing entries provide examples.

3. Restart Studio and select the solver.

Install any dependencies required by your wrapper in the Python environment used to launch Studio, then restart Studio. Select **PORTAL** as the algorithm **Source** and choose your solver from the **Algorithm** list. By default, the displayed name comes from the wrapper's filename: `MySolver.py` appears as **MySolver**.

If the solver does not appear, check that the file imports successfully and defines its own `BaseAlgorithm` subclass with a `run` method. Studio skips wrappers that fail to import.

4. Check the integration.

Start with a small evaluation budget and confirm that the solver runs, respects the budget, and produces results and a convergence plot. Then check that changing its GUI parameters affects the settings used by the algorithm. The existing wrappers in `algorithms/` provide examples of complete solver implementations.

References

[1] D. Yazdani, M. Peng, D. Yazdani, S. F. Yazdi, M. N. Omidvar, Y. Sun, T. T. Nguyen, C. Li, and X. Li, "PORTAL: Controllable landscape generator for continuous optimization—part I: Framework," arXiv:2512.00288, 2025. [Online]. Available: https://arxiv.org/abs/2512.00288

[2] R. P. Prager and H. Trautmann, "Pflacco: Feature-based landscape analysis of continuous and constrained optimization problems in python," *Evolutionary Computation*, vol. 32, no. 3, pp. 211–216, 2024.

[3] G. Vrbančič, L. Brezočnik, U. Mlakar, D. Fister, and I. Fister, "NiaPy: Python microframework for building nature-inspired algorithms," *Journal of Open Source Software*, vol. 3, no. 23, p. 613, 2018.

[4] J. Blank and K. Deb, "pymoo: Multi-objective optimization in python," *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.